

# Documentación técnica

## Formularios de contacto y agenda del sitio web

Cómo funcionan el formulario de mensajes y el de agendar cita, qué tecnologías se usan y por qué, dónde se guardan las llaves secretas, y a dónde llegan los correos y las citas.

|           |                                                                                                                       |
|-----------|-----------------------------------------------------------------------------------------------------------------------|
| Sitio     | <a href="http://www.wearevectis.com">www.wearevectis.com</a>                                                          |
| Hosting   | Vercel · Repositorio: <a href="https://github.com/marlonxar/Vectis-Frontend">github.com/marlonxar/Vectis-Frontend</a> |
| Stack     | Angular (SPA) + funciones serverless (/api) en Vercel                                                                 |
| Documento | Interno · Junio 2026                                                                                                  |

# 1. Resumen general

El sitio web es una aplicación Angular estática alojada en Vercel. En la sección **Contacto** hay dos formularios interactivos:

- **Enviar mensaje** — envía un correo a Vectis con la consulta de la persona.
- **Agendar cita** — reserva una reunión real en el calendario, en 4 pasos.

Como el sitio es estático (no tiene un servidor propio corriendo todo el tiempo), las operaciones que necesitan **llaves secretas** se hacen mediante **funciones serverless**: pequeños archivos en la carpeta `/api` que Vercel ejecuta bajo demanda. Ahí viven los secretos, nunca en el navegador.

Idea clave: el navegador (lo que ve el visitante) nunca toca una llave secreta. Cuando hace falta un secreto, el navegador le pide a una función `/api` que haga la operación de forma segura.

## 2. Tecnologías que se usan y por qué

| Tecnología               | Para qué                                                         | Por qué esta                                                                                                                    |
|--------------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Angular                  | Construye toda la página web (la interfaz).                      | Es el framework con el que está hecho el sitio.                                                                                 |
| Vercel (serverless /api) | Ejecuta código seguro que guarda las llaves secretas.            | No hay que mantener un servidor; se publica junto al sitio. Es lo más liviano.                                                  |
| Cal.com                  | Motor de agenda: disponibilidad real y creación de citas.        | Gratis y open-source, con API de reservas y sincronización nativa con Google Calendar. (Calendly cobra por su API de reservas.) |
| Web3Forms                | Entrega por correo lo que se envía en el formulario de mensajes. | Gratis y sin servidor de correo propio. Hace de 'cartero'.                                                                      |
| Cloudflare Turnstile     | Verifica que quien envía es una persona, no un bot.              | Anti-bot invisible (sin rompecabezas ni popups) y gratis e ilimitado.                                                           |
| Honeypot                 | Trampa extra anti-bots (campo oculto).                           | Gratis y sin fricción; descarta bots en silencio.                                                                               |

## 3. Formulario «Enviar mensaje»

### Campos y validación

Campos obligatorios (marcados con \*): Nombre, Email, Empresa, Presupuesto, Asunto y Mensaje, además del check de aceptación de Política de Privacidad y Términos. El botón **Enviar** permanece deshabilitado hasta que todos los obligatorios sean válidos, el check esté marcado y Cloudflare Turnstile confirme que es una persona.

### Qué pasa al enviar (paso a paso)

1. La persona completa el formulario; el widget de Turnstile la verifica de forma invisible y entrega un 'ticket' (token).
2. El navegador envía ese token a la función `/api/contact`.
3. `/api/contact` revisa el honeypot y valida el token contra Cloudflare usando la llave secreta `CF_TURNSTILE_SECRET`. Si es bot, se rechaza.

- 4. Si es persona, el **navegador** envía los datos del formulario directamente a Web3Forms con la llave pública de acceso.
- 5. Web3Forms entrena el mensaie por correo.

¿Por qué el correo lo envía el navegador y no el servidor? Porque Web3Forms está protegido por Cloudflare y bloquea las peticiones desde IPs de servidores (como las de Vercel) con un reto 'Just a moment'. Los navegadores sí son aceptados (es su uso normal). Por eso el servidor solo valida el captcha y el navegador hace la entrega.

### A dónde llegan los correos

Al buzón configurado en la cuenta de **Web3Forms**: actualmente **vectisauto@gmail.com**. Para cambiarlo, se modifica el correo asociado a la access key en el panel de Web3Forms (o se crea una nueva access key con otro correo).

## 4. Formulario «Agendar cita»

Es un asistente de 4 pasos con diseño propio, conectado a Cal.com por debajo:

- **Paso 1 — Servicio:** la persona elige el servicio de interés.
- **Paso 2 — Fecha y hora:** el calendario muestra la **disponibilidad real**. El navegador pide los horarios a `/api/cal-slots`, que consulta Cal.com (API v2) con `CAL_API_KEY` y `CAL_EVENT_TYPE_ID` y devuelve solo los espacios libres.
- **Paso 3 — Datos:** nombre, email, empresa y mensaje.
- **Paso 4 — Confirmar:** muestra un resumen. Al confirmar, el navegador llama a `/api/cal-book`, que crea la reserva real en Cal.com (API v2).

El servicio elegido, la empresa y el mensaje se guardan en las notas de la reserva, así que aparecen en la **descripción del evento** del calendario.

### A dónde llegan las citas

La reserva se crea en **Cal.com** → **Bookings** y se sincroniza automáticamente al **Google Calendar** conectado en Cal.com. Cal.com envía los correos de confirmación tanto a Vectis como a la persona que agenda. (Requisito: tener Google Calendar conectado en Cal.com → Settings → Apps/Calendars.)

## 5. Variables de entorno (dónde se guardan las llaves)

Las llaves **secretas** se almacenan en **Vercel** → **Project** → **Settings** → **Environment Variables**, en el entorno **Production** (marcadas como *Sensitive*). Nunca van en el código ni en el navegador. Importante: al agregar o cambiar una variable hay que hacer un **redesploy** para que tome efecto.

| Variable            | Tipo             | Para qué                                                                      |
|---------------------|------------------|-------------------------------------------------------------------------------|
| CAL_API_KEY         | Secreta (Vercel) | Autentica con Cal.com para leer disponibilidad y crear reservas.              |
| CAL_EVENT_TYPE_ID   | No secreta       | ID del evento a reservar (30 min). Valor: 6002107 (por defecto en el código). |
| CF_TURNSTILE_SECRET | Secreta (Vercel) | Valida el captcha de Turnstile en el servidor.                                |

|               |                    |                                                                                            |
|---------------|--------------------|--------------------------------------------------------------------------------------------|
| WEB3FORMS_KEY | Pública (opcional) | Access key de Web3Forms. Está como respaldo en el código; puede ponerse también en Vercel. |
|---------------|--------------------|--------------------------------------------------------------------------------------------|

### Llaves públicas (seguras de exponer, viven en el código del frontend)

| Llave                | Dónde                | Nota                                                |
|----------------------|----------------------|-----------------------------------------------------|
| TURNSTILE_SITE_KEY   | contact.component.ts | Llave pública del widget de Turnstile.              |
| Web3Forms access key | contact.component.ts | Diseñada para ser pública (uso desde el navegador). |

Nota: que una llave sea pública no es un descuido. Turnstile (con su secret en el servidor) y el honeypot son los que aportan la seguridad real; las llaves públicas por sí solas no permiten abuso significativo.

## 6. Archivos clave del proyecto

| Archivo                                         | Responsabilidad                                                                    |
|-------------------------------------------------|------------------------------------------------------------------------------------|
| src/app/features/contact/contact.component.ts   | Lógica de ambos formularios: validación, Turnstile, llamadas a /api y a Web3Forms. |
| src/app/features/contact/contact.component.html | Interfaz de los formularios (campos, pasos, calendario, botones).                  |
| api/contact.js                                  | Valida honeypot + Turnstile del formulario de mensajes.                            |
| api/cal-slots.js                                | Devuelve la disponibilidad real desde Cal.com.                                     |
| api/cal-book.js                                 | Crea la reserva en Cal.com (con notas/servicio en la descripción).                 |
| vercel.json                                     | Reglas de rutas: SPA, excluyendo /api para que las funciones respondan.            |

## 7. Mantenimiento (cómo cambiar cosas)

- **Cambiar a dónde llegan los correos:** en el panel de Web3Forms, cambiar el correo de la access key (o usar una nueva).
- **Cambiar a dónde llegan las citas:** es el Google Calendar conectado en Cal.com; se gestiona desde Cal.com → Settings → Calendars.
- **Cambiar el tipo de evento de la cita:** actualizar la variable CAL\_EVENT\_TYPE\_ID en Vercel y redeploy.
- **Rotar una llave secreta:** generar la nueva en el servicio (Cal.com / Cloudflare), actualizar la variable en Vercel y hacer redeploy.
- **Tras cualquier cambio de variable en Vercel:** siempre redeploy, si no, no toma efecto.